Laboratório Avaliativo Prático: Construindo uma Arquitetura de Microsserviços

Modalidade

Laboratório em dupla

Objetivos

Ao final desta atividade o estudante deverá ser capaz de:

- implementar microsserviços independentes;
- utilizar comunicação síncrona via REST;
- utilizar comunicação assíncrona via RabbitMQ;
- compreender o conceito de banco por serviço;
- executar serviços utilizando Docker Compose;
- analisar o comportamento da arquitetura diante de falhas;
- compreender escalabilidade horizontal;
- utilizar logs para observabilidade e diagnóstico.

Observação: modificações no projeto são aceitas devido a incompletude e/ou falhas conceituais na descrição do mesmo.

Cenário

Uma empresa está desenvolvendo uma plataforma de comércio eletrônico.

A solução deverá permitir:

- consulta de produtos;
- reserva de estoque;
- criação de pedidos;
- processamento de pagamentos.

A arquitetura deverá ser composta pelos seguintes microsserviços:

```
Estoque Service
Pedido Service
Pagamento Service
```

Organização da Dupla

Importante: ambos os integrantes devem compreender e ser capazes de explicar todos os serviços e mecanismos implementados.

Aluno A: Responsável inicialmente por: Pedido Service

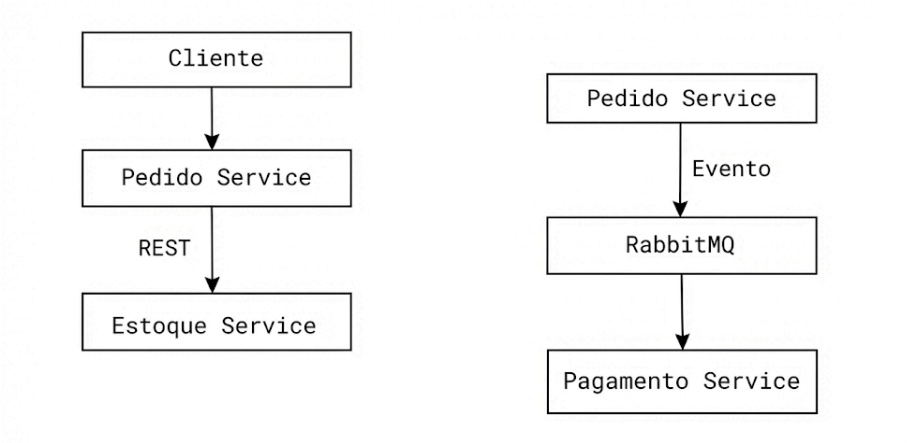
Aluno B: Responsável inicialmente por: Estoque Service

Trabalho Conjunto

Após a implementação inicial dos dois serviços, a dupla deverá trabalhar conjuntamente nas seguintes etapas:

```
integração entre Pedido e Estoque;
implementação do Pagamento Service;
configuração do RabbitMQ;
configuração do Docker Compose;
testes de falhas e recuperação;
escalabilidade;
observabilidade.
```

Arquitetura Final



Estrutura Esperada

```
microservices-lab

      ├── pedido-service
      |          pedido-db
      ├── estoque-service
      |          estoque-db
      ├── pagamento-service
      |          pagamento-db
      └── docker-compose.yml
```

É proibido que um serviço consulte diretamente o banco de outro serviço.

Durante a atividade, a dupla deverá demonstrar que:

1. cada serviço possui seu próprio banco de dados;
2. o Pedido Service não acessa diretamente o banco do Estoque Service;
3. o Pagamento Service não acessa diretamente o banco do Pedido Service;
4. a comunicação entre os serviços ocorre por meio das interfaces e mecanismos definidos neste laboratório.

Tecnologias

Não há impedimento sobre o uso de outras tecnologias/linguagens.

Sugeridas

1. Java 17+
2. Spring Boot
3. Spring Data JPA
4. RabbitMQ
5. Docker Compose

Ferramentas de Teste

- Postman
- Insomnia
- cURL

Etapa 1 – Implementando o Estoque Service

Responsável: Aluno B

Modelo de Dados

```
CREATE TABLE produto (
      id BIGINT PRIMARY KEY,
      nome VARCHAR(100),
      quantidade INTEGER
);
```

Dados Iniciais

```
1     Notebook    10
2     Mouse       50
3     Teclado     20
```

Entidade

```
@Entity
public class Produto {
      @Id
```

```
        private Long id;

        private String nome;

        private Integer quantidade;
    }
```

Endpoint 1 - Consultar todos os produtos

```
GET /produtos
```

Resposta:

```
[
      {
       "id": 1,
       "nome": "Notebook",
       "quantidade": 10
      }
]
```

Endpoint 2 - Consultar produto

```
GET /produtos/{id}
```

Endpoint 3 - Reservar estoque

```
PUT /produtos/{id}/reservar
```

Body:

```
{
 "quantidade": 2
}
```

Regra

Se o produto existir e possuir quantidade suficiente:

- reduzir a quantidade disponível
- retornar HTTP 200

Se o produto existir, mas a quantidade solicitada for maior que a disponível:

- não alterar o estoque;
- retornar HTTP 409

  Resposta:

```
{
 "mensagem": "Estoque insuficiente"
}
```

Se o produto não existir:

- não alterar o estoque;
- retornar HTTP 404

  Resposta:

```
{
 "mensagem": "Produto inexistente?"
}
```

Etapa 2 – Implementando o Pedido Service

Responsável: Aluno A

Modelo de Dados

```
CREATE TABLE pedido (
      id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
      produto_id BIGINT,
      quantidade INTEGER,
      status VARCHAR(50)
);
```

Entidade

```
@Entity
```

```
public class Pedido {
	@Id
	@GeneratedValue(strategy = GenerationType.IDENTITY)
	private Long id;
	private Long produtoId;
	private Integer quantidade;
	private String status;
}
```

Status

AGUARDANDO_PAGAMENTO
PAGO
REJEITADO

Estado inicial

- Todo novo pedido deverá ser criado inicialmente com o status AGUARDANDO_PAGAMENTO.
- Após o processamento do pagamento, o status deverá ser alterado para PAGO ou REJEITADO.

Endpoint - Criar pedido

```
POST /pedidos
```

Body:

```
{
 "produtoId": 1,
 "quantidade": 2
}
```

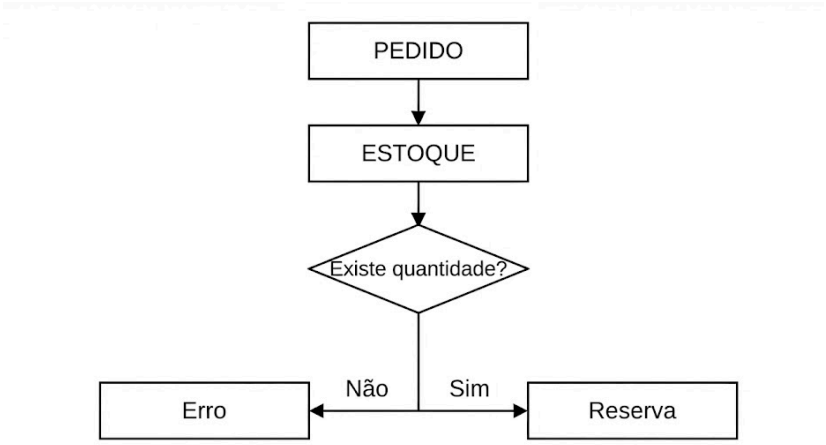
Endpoint

```
GET /pedidos
GET /pedidos/{id}
```

Etapa 3 – Integração REST

Nesta etapa é preciso integrar os serviços.

Fluxo:



Regra de Negócio

Se houver estoque:

1. Reservar estoque
2. Criar pedido
3. Publicar evento

Se não houver estoque:

- NÃO criar pedido
- NÃO publicar evento
- RETORNAR erro

Experimento de consistência

A implementação deverá considerar que as operações de reserva de estoque e criação do pedido pertencem a serviços e bancos de dados diferentes.

Simule uma falha após a reserva do estoque e antes da criação do pedido.

Responda:

1. O que aconteceu com o estoque?
2. O pedido foi criado?
3. Existe uma transação única envolvendo os dois serviços?
4. Como o sistema poderia desfazer a reserva realizada?
5. Que mecanismo poderia ser utilizado para realizar essa compensação?

Exemplo RestTemplate

```
restTemplate.exchange(
      url,
      HttpMethod.PUT,
      entity,
      Void.class
);
```

Etapa 4 – Docker Compose

Criar o arquivo: docker-compose.yml

Configuração

```
services:
  rabbitmq:
    image: rabbitmq:3.13-management
    ports:
      - "5672:5672"
      - "15672:15672"
    healthcheck:
      test: ["CMD", "rabbitmq-diagnostics", "-q", "ping"]
      interval: 10s
      timeout: 5s
      retries: 10

  estoque-db:
    image: postgres:16
    environment:
      POSTGRES_DB: estoque
      POSTGRES_USER: estoque
```

```
    POSTGRES_PASSWORD: estoque
  volumes:
    - estoque-data:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U estoque -d estoque"]
    interval: 5s
    timeout: 3s
    retries: 10

pedido-db:
  image: postgres:16
  environment:
    POSTGRES_DB: pedido
    POSTGRES_USER: pedido
    POSTGRES_PASSWORD: pedido
  volumes:
    - pedido-data:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U pedido -d pedido"]
    interval: 5s
    timeout: 3s
    retries: 10

pagamento-db:
  image: postgres:16
  environment:
    POSTGRES_DB: pagamento
    POSTGRES_USER: pagamento
    POSTGRES_PASSWORD: pagamento
  volumes:
    - pagamento-data:/var/lib/postgresql/data
  healthcheck:
```

```
    test: ["CMD-SHELL", "pg_isready -U pagamento -d pagamento"]
    interval: 5s
    timeout: 3s
    retries: 10

estoque-service:
  build: ./estoque-service
  ports:
    - "8081:8080"
  environment:
    SPRING_DATASOURCE_URL: jdbc:postgresql://estoque-db:5432/estoque
    SPRING_DATASOURCE_USERNAME: estoque
    SPRING_DATASOURCE_PASSWORD: estoque
  depends_on:
    estoque-db:
      condition: service_healthy

pedido-service:
  build: ./pedido-service
  ports:
    - "8080:8080"
  environment:
    SPRING_DATASOURCE_URL: jdbc:postgresql://pedido-db:5432/pedido
    SPRING_DATASOURCE_USERNAME: pedido
    SPRING_DATASOURCE_PASSWORD: pedido
    SPRING_RABBITMQ_HOST: rabbitmq
    ESTOQUE_SERVICE_URL: http://estoque-service:8080
  depends_on:
    pedido-db:
      condition: service_healthy
    rabbitmq:
      condition: service_healthy
```

```
      estoque-service:
        condition: service_started


  pagamento-service:
    build: ./pagamento-service
    expose:
      - "8080"
    environment:
      SPRING_DATASOURCE_URL:
jdbc:postgresql://pagamento-db:5432/pagamento
      SPRING_DATASOURCE_USERNAME: pagamento
      SPRING_DATASOURCE_PASSWORD: pagamento
      SPRING_RABBITMQ_HOST: rabbitmq
    depends_on:
      pagamento-db:
        condition: service_healthy
      rabbitmq:
        condition: service_healthy


volumes:
  estoque-data:
  pedido-data:
  pagamento-data:
```

Executar

```
docker compose up
```

Se preciso, você pode verificar os logs para confirmar se os serviços estão disponíveis.

```
docker compose logs
```

Verificar

Pedido consegue acessar Estoque?

Etapa 5 – Introduzindo RabbitMQ

Adicionar comunicação assíncrona.

```
Exchange: pedidos.exchange

Queue: pedido.criado

Routing Key: pedido.criado
```

Experimento

Acesse a interface de gerenciamento do RabbitMQ e observe:

1. o Exchange pedidos.exchange;
2. a Queue pedido.criado;
3. a quantidade de mensagens na fila;
4. o consumidor conectado à fila.

Durante a execução das próximas etapas, observe como esses valores se modificam.

Responda:

1. Por que o Pedido Service publica em um Exchange em vez de enviar diretamente para uma Queue?
2. Qual é a diferença entre Exchange, Queue e Consumer?
3. O Pedido Service sabe quem consumirá o evento?

Evento

```
{

  "pedidoId": 10,

  "produtoId": 1,

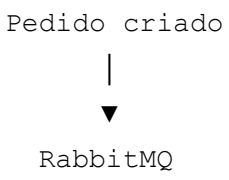
  "quantidade": 2,

  "correlationId": "b1f6c1a2-5c5e-4c39-9f52-0d3b0d2c9d10"

}
```

Publicação

Após criar o pedido:



Exemplo

```
rabbitTemplate.convertAndSend(
      "pedidos.exchange",
      "pedido.criado",
      evento
);
```

Etapa 6 – Implementando o Pagamento Service

Responsabilidade: Processar pagamentos

Ao receber um evento pedido.criado, o Pagamento Service deverá:

1. registrar o pagamento associado ao pedido;
2. determinar aleatoriamente se o pagamento será aprovado ou rejeitado;
3. persistir o resultado no seu banco de dados;
4. registrar a operação no log.

O processamento deverá ocorrer independentemente de o Pedido Service estar disponível.

Modelo de Dados

```
CREATE TABLE pagamento (

      id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,

      pedido_id BIGINT,

      status VARCHAR(50)

);
```

Consumidor

```
@RabbitListener(
      queues = "pedido.criado"
)

public void consumir(PedidoEvento evento) {

}
```

Regra

Aprovar ou rejeitar o pagamento aleatoriamente, com aproximadamente 80% de aprovação e 20% de rejeição.

Fluxo

```
Pedido
   │
   ▼
RabbitMQ
   │
   ▼
Pagamento
```

Etapa 7 – Teste Funcional

Criar um pedido:

```
POST /pedidos
```

Verificar

1. o estoque foi atualizado;
2. o pedido foi criado com status AGUARDANDO_PAGAMENTO;
3. o evento pedido.criado foi publicado;
4. o Pagamento Service consumiu o evento;
5. o pagamento foi registrado no banco do Pagamento Service;
6. o resultado do pagamento foi registrado no log.

Etapa 8 – Simulação de Falha

Parar o serviço:

```
docker compose stop pagamento-service
```

Criar Novos Pedidos

```
POST /pedidos
```

Responder

1. O pedido foi criado?
2. O estoque foi atualizado?
3. O sistema inteiro parou?
4. A mensagem foi perdida? Apresente evidências obtidas a partir da fila do RabbitMQ e dos logs.

Etapa 9 – Recuperação

Subir novamente:

```
docker compose start pagamento-service
```

Verificar

1. se as mensagens geradas durante a indisponibilidade do Pagamento Service permanecem disponíveis na fila;
2. iniciar novamente o Pagamento Service;
3. verificar se as mensagens pendentes são consumidas;
4. verificar se cada pedido foi processado;
5. verificar os registros no banco do Pagamento Service.

Responder

1. O processamento precisou ser repetido manualmente?
2. O Pedido Service precisou aguardar o Pagamento Service?
3. O que aconteceu com as mensagens enquanto o consumidor estava indisponível?

Etapa 10 – Escalabilidade

Executar:

```
docker compose up --scale pagamento-service=2
```

Criar Diversos Pedidos

Observar logs:

```
pagamento-service-1

pagamento-service-2
```

Responder

- As mensagens foram distribuídas?
- Apenas uma instância processou cada mensagem?
- Observe o comportamento das duas instâncias de Pagamento. Em seguida, discuta quais características da arquitetura permitem que apenas esse serviço seja escalado independentemente dos demais.
- Em quais circunstâncias o Estoque Service também precisaria ser escalado?

Etapa 11 – Observabilidade

Adicionar logs.

O Pedido Service deverá gerar um correlationId único para cada requisição e propagá-lo para os demais serviços e mensagens.

```
UUID.randomUUID().toString()
```

Pedido Service

```
log.info(
  "correlationId={} Pedido {} criado",
  correlationId, pedidoId
);
```

Estoque Service

```
log.info(
  "correlationId={} Produto {} reservado",
  correlationId, produtoId
);
```

RabbitMQ

```
log.info(
  "correlationId={} Evento publicado {}",
  correlationId, pedidoId
);
```

Pagamento Service

```
log.info(
  "correlationId={} Pagamento aprovado {}",
  correlationId, pedidoId
```

```
);
```

Investigação de Incidente

Considere que o usuário informou que o **Pedido #17 não foi concluído**.

A dupla deverá utilizar os logs dos três microsserviços e o RabbitMQ para reconstruir o fluxo de processamento do pedido.

```
docker compose logs pedido-service

docker compose logs estoque-service

docker compose logs pagamento-service
```

Utilize o `pedidoId` e o `correlationId` para acompanhar o processamento.

Perguntas

1. O pedido foi criado?
2. O estoque foi reservado?
3. O evento pedido.criado foi publicado?
4. O Pagamento Service recebeu o evento?
5. O pagamento foi processado?
6. Em qual etapa ocorreu o problema?
7. Qual evidência nos logs permite identificar a etapa da falha?
8. O que aconteceu com a mensagem no RabbitMQ?

Etapa 12 - Atualização assíncrona do pedido

Implementar a atualização automática do status do pedido. O Pagamento Service deverá publicar um evento após processar o pagamento:

```
pagamento.processado
```

O evento deverá informar:

```
{
```

```
  "pedidoId": 10,
  "status": "APROVADO",
  "correlationId": "b1f6c1a2-5c5e-4c39-9f52-0d3b0d2c9d10"
}
```

ou

```
{
  "pedidoId": 10,
  "status": "REJEITADO",
  "correlationId": "b1f6c1a2-5c5e-4c39-9f52-0d3b0d2c9d10"
}
```

Fluxo:



O Pedido Service deverá consumir o evento e atualizar o status do pedido para PAGO ou REJEITADO.

Importante: o Pagamento Service não deverá acessar diretamente o banco do Pedido Service.

Teste

Execute pedidos suficientes para observar pelo menos:

- um pagamento aprovado;
- um pagamento rejeitado.

Verifique se o status do pedido foi atualizado corretamente nos dois casos.

Entrega

Elaborar um documento com os itens abaixo que devem ter sido elaborados pela dupla.

Apenas uma entrega é suficiente, todavia é necessário o nome de ambos no documento.

Parte 1

Diagrama da arquitetura.

Parte 2

Código-fonte dos serviços.

Parte 3

Arquivo: docker-compose.yml

Parte 4

Prints demonstrando:

- criação do pedido;
- reserva de estoque;
- publicação da mensagem;
- processamento do pagamento.

Parte 5

Respostas das perguntas realizadas em todas as etapas:

[2026/2] IMPORTANTE: essa atividade faz parte da avaliação, correspondendo a 40% da nota.